Introdução à Linguagem Assembly

Como o Software Encontra o Hardware



Arquitetura e Organização de Computadores (AOC)

Gemini Notebook

O processador entende um programa em C?

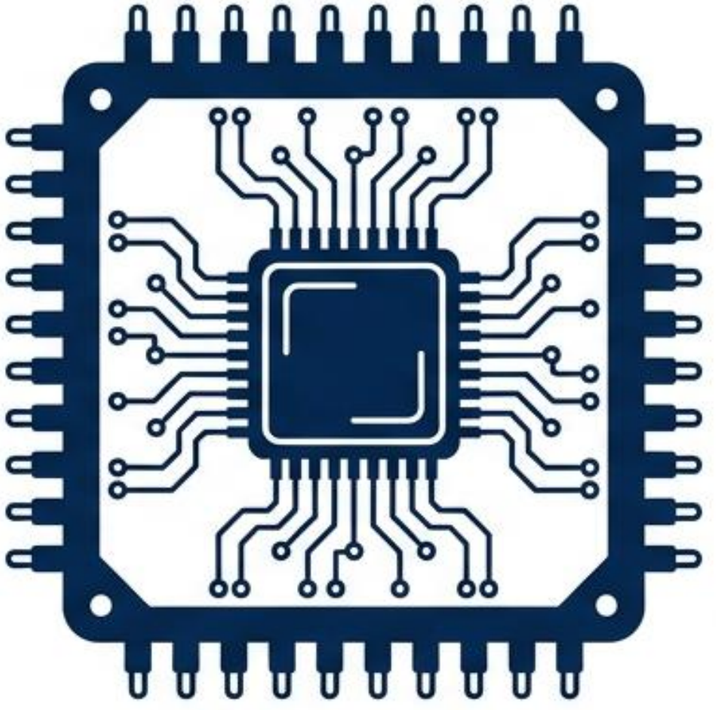
Não.

```c
#include <stdio.h>

int main() {
  printf("Olá, Mundo!\n");
  return 0;
}
```

programa.c





Hardware (CPU)

Processadores modernos são compostos por bilhões de transistores. Eles não compreendem texto, lógica de alto nível ou variáveis. Eles operam exclusivamente através de sinais elétricos, representados matematicamente como Linguagem de Máquina (Binário).

Gemini Notebook

O Caminho até o Hardware



O Assembly é a representação legível por humanos da linguagem de máquina. Ele possui uma relação direta e de 1-para-1 com o conjunto de instruções (ISA) do processador.

Gemini Notebook

Por que utilizamos Mnemônicos?

Em vez de memorizar códigos binários opacos (Opcodes), utilizamos **mnemônicos** — palavras curtas que mapeiam diretamente as operações eletrônicas da CPU.

O que a máquina lê

```
B8 05 00 00 00
10111000 00000101....
```



O que o humano lê

```
mov eax, 5
```

Gemini Notebook

A Anatomia de uma Instrução



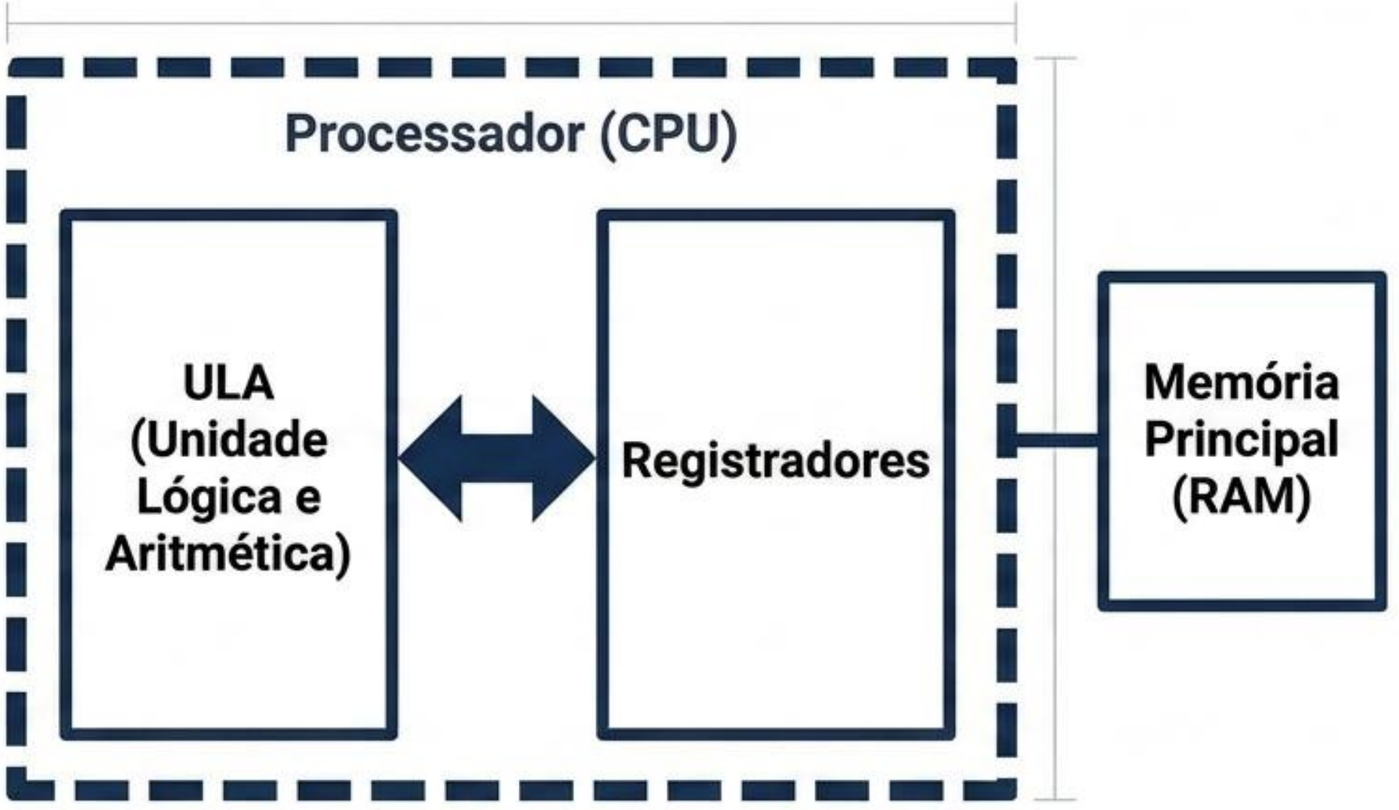
Gemini Notebook

O Palco de Execução

Para programar em Assembly, precisamos enxergar dentro do chip.

A ULA faz os cálculos.

Onde ela guarda os números durante a conta? Nos Registradores.



CPU Dashboard

Gemini Notebook

Conhecendo os Atores: Registradores

São pequenas áreas de rascunho, extremamente rápidas, embutidas no próprio processador. Pense neles como as variáveis exclusivas do hardware.



Gemini Notebook

O Vocabulário Básico: Movimentação e Aritmética

Mnemônico	Significado	Exemplo (Sintaxe Intel)
MOV	Copiar (Mover) dado	`mov eax, 5` (EAX recebe 5)
ADD	Somar	`add eax, ebx` (EAX = EAX + EBX)
SUB	Subtrair	`sub ebx, 1` (EBX = EBX – 1)
INC	Incrementar (+1)	`inc ecx` (ECX = ECX + 1)
DEC	Decrementar (-1)	`dec edx` (EDX = EDX – 1)

Gemini Notebook

Seja a CPU: Execução Passo a Passo

Tempo 1

`mov eax, 5`

EAX **5**

EBX **0**

Tempo 2

`mov ebx, 3`

EAX **5**

EBX **3**

Tempo 3

`add eax, ebx`

EAX **8**

EBX **3**



Gemini Notebook

Desafio: O que acontecerá agora?

```
1. mov ecx, 10
2. sub ecx, 2
3. inc ecx
```

CPU Dashboard

ECX

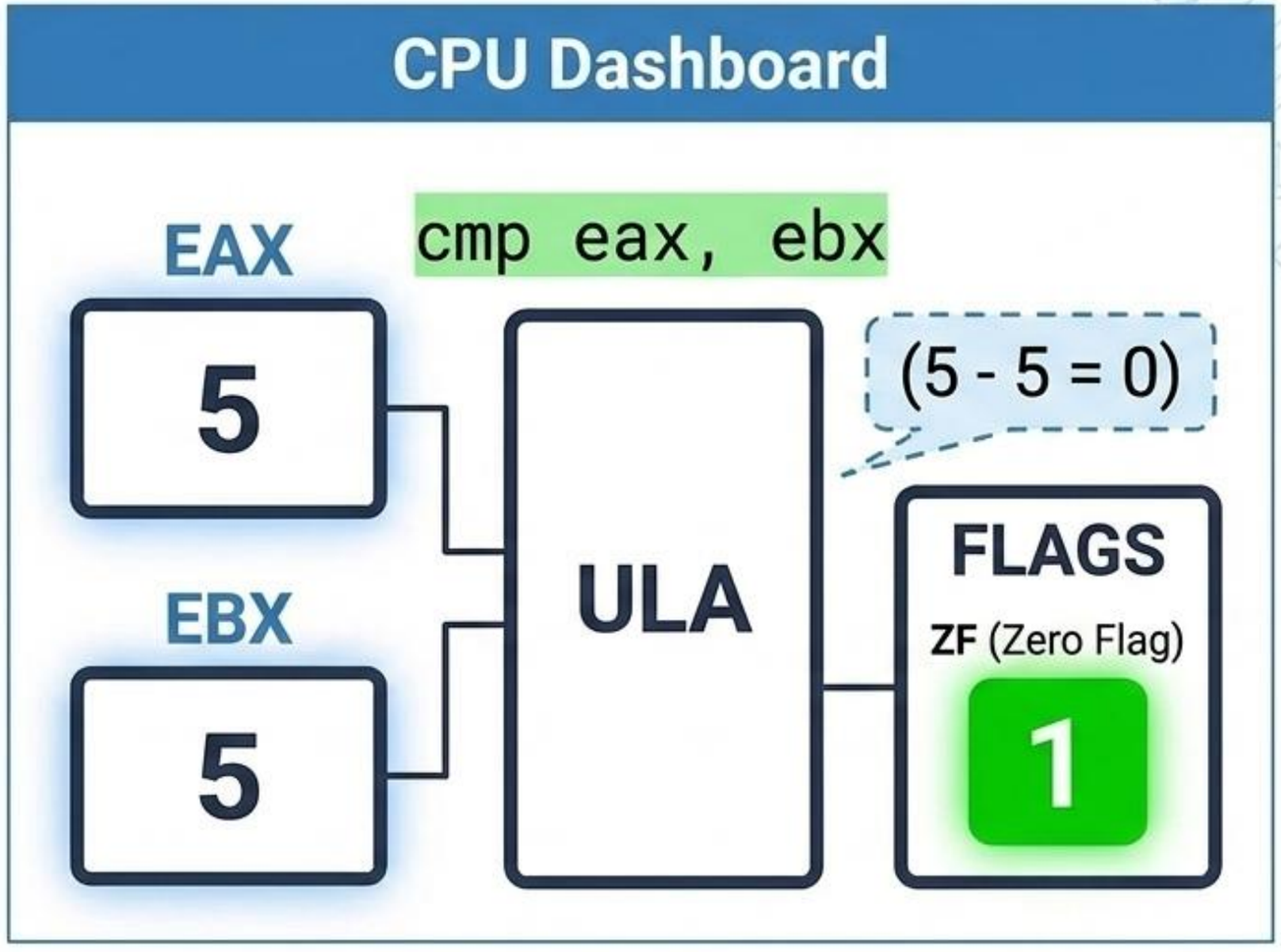
?

Qual será o valor final dentro do registrador ECX ao término destas três instruções? (Pense como a ULA!)

Gemini Notebook

Tomando Decisões: Comparações e Flags

- O processador não entende "Verdadeiro" ou "Falso".
- A instrução **CMP** (Compare) faz uma subtração subtração invisível na ULA.
- Se o resultado for zero (indicando que os números são iguais), ela acende uma luz especial chamada **Zero Flag** (ZF).

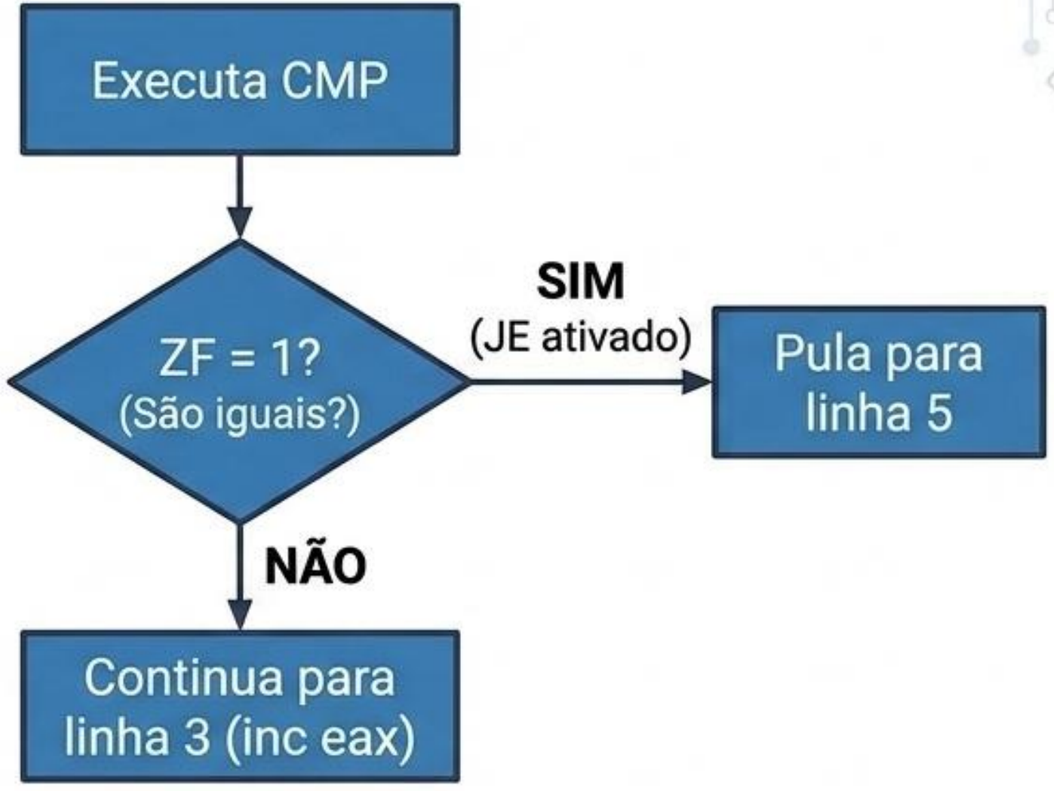


Gemini Notebook

Mudando a Rota: Desvios (Jumps)

JMP pula para outra linha de código incondicionalmente.
JE (**Jump if Equal**) olha para a Zero Flag: ele só pula se a luz da ZF estiver acesa.

```
1.  cmp eax, ebx
2.  je CAMINHO_IGUAL
3.  inc eax
4.  ...
5.  CAMINHO_IGUAL:
6.  mov eax, 0
```



Gemini Notebook

A Pedra de Roseta: C vs. Assembly

Os compiladores transformam estruturas de alto nível familiares em blocos básicos de movimentação, cálculo e desvio.

Conceito	Linguagem C	Assembly Conceitual
Atribuição	`x = 5;`	`mov eax, 5`
Incremento	`x++;`	`inc eax`
Soma	`z = x + y;`	`mov eax, x` `add eax, y`
Condição (IF)	`if (x == y) { ... }`	`cmp eax, ebx` `je IGUAL`

Gemini Notebook

Assembly no Mundo Real (Prática Linux)

No laboratório, usaremos essas ferramentas reais para gerar Assembly, inspecionar binários e “congelar o tempo” observando os registradores em execução.

```
$ gcc -O0 -S programa.c
# -> Traduz o código C e gera o arquivo Assembly (.s)
$ objdump -d executavel
# -> Inspeciona o código de máquina e os mnemônicos do binário
$ gdb executavel
# -> O Debugger: permite executar passo a passo e ler os
registradores (info registers)
```

Gemini Notebook

O Fio Condutor



“Todo o software do mundo, por mais complexo que seja, se reduz a movimentar dados e fazer operações simples nestas minúsculas áreas de hardware.”

Até agora, nós fomos a CPU. No próximo laboratório, vamos observar uma CPU real executando essas instruções.

Gemini Notebook